

## Unit-2 Relational Model

### GTU Syllabus:

#### Relational Model

Structure of relational databases, Database Schema, Keys, Schema Diagrams, Relational algebra operators, relational algebra queries.

#### Relational Model concept:

Relational model can represent as a table with columns and rows. Each row is known as a tuple. Each table of the column has a name or attribute.

**Domain:** It contains a set of atomic values that an attribute can take.

**Attribute:** It contains the name of a column in a particular table. Each attribute  $A_i$  must have a domain,  $dom(A_i)$

**Relational instance:** In the relational database system, the relational instance is represented by a finite set of tuples. Relation instances do not have duplicate tuples.

**Relational schema:** A relational schema contains the name of the relation and name of all columns or attributes.

**Relational key:** In the relational key, each row has one or more attributes. It can identify the row in the relation uniquely.

#### Example: STUDENT Relation

| NAME   | ROLL_NO | PHONE_NO    | ADDRESS   | AGE |
|--------|---------|-------------|-----------|-----|
| Ram    | 14795   | 7305758992  | Noida     | 24  |
| Shyam  | 12839   | 9026288936  | Delhi     | 35  |
| Laxman | 33289   | 8583287182  | Gurugram  | 20  |
| Mahesh | 27857   | 7086819134  | Ghaziabad | 27  |
| Ganesh | 17282   | 9028 9i3988 | Delhi     | 40  |

- In the given table, NAME, ROLL\_NO, PHONE\_NO, ADDRESS, and AGE are the attributes.
- The instance of schema STUDENT has 5 tuples.
- $t_3 = \langle \text{Laxman}, 33289, 8583287182, \text{Gurugram}, 20 \rangle$

## Properties of Relations

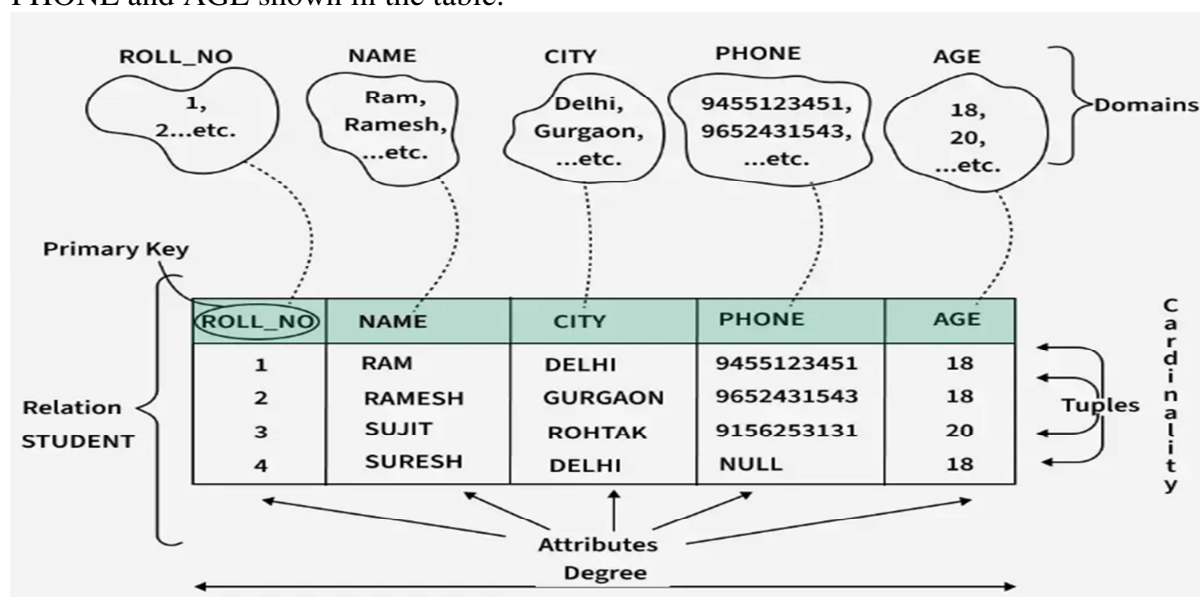
- Name of the relation is distinct from all other relations.
- Each relation cell contains exactly one atomic (single) value
- Each attribute contains a distinct name
- Attribute domain has no significance
- tuple has no duplicate value
- Order of tuple can have a different sequence

## Structure of relational databases:

The Relational Model organizes data using tables (relations) consisting of rows and columns.

- The relational model represents how data is stored and managed in Relational Databases where data is organized into tables, each known as a relation.
- Each row of a table represents an entity or record and each column represents a particular attribute of that entity.
- The relational model transforms conceptual designs from ER diagrams into implementable structures. These structures are used in relational database systems like Oracle SQL and MySQL.

**Example:** Consider a relation STUDENT with attributes ROLL\_NO, NAME, ADDRESS, PHONE and AGE shown in the table.



Relational Model

## **Key Terms in the Relational Model**

1. **Attribute:** Attributes are the properties that define an entity. For example, ROLL\_NO, NAME, ADDRESS etc.
2. **Relation Schema:** A relation schema defines the structure of the relation and represents the name of the relation with its attributes. For example, STUDENT (ROLL\_NO,

NAME, ADDRESS, PHONE and AGE) is the relation schema for STUDENT. If a schema has more than 1 relation it is called Relational Schema.

3. **Tuple:** A Tuple represents a row in a relation. Each tuple contains a set of attribute values that describe a particular entity. For example, (1, RAM, DELHI, 9455123451, 18) is a tuple in the STUDENT table.
4. **Relation Instance:** The set of tuples of a relation at a particular instance of time is called a relation instance. It can change whenever there is an insertion, deletion or update in the database.
5. **Degree:** The number of attributes in the relation is known as the degree of the relation. For example, The STUDENT relation has a degree of 5, as it has 5 attributes.
6. **Cardinality:** The number of tuples in a relation is known as cardinality. For example, The STUDENT relation defined above has cardinality 4.
7. **NULL Values:** The value which is not known or unavailable is called a NULL value. It is represented by NULL. For example, PHONE of STUDENT having ROLL\_NO 4 is NULL.

### Types of Keys in the Relational Model:

#### 1. Super Key

A **Super Key** is any set of one or more attributes (columns) that, when taken together, can uniquely identify every tuple (row) in a relation (table). A super key may contain extra attributes that are not strictly necessary for uniqueness.

- **Definition:** Any set of attributes that can uniquely identify a row.
- **Example (in Employee table):**
  - {EmployeeID}
  - {SSN}
  - {EmployeeID, Name} (contains extra, non-essential attributes)
  - {EmployeeID, SSN, Name, DeptID} (contains all attributes)

#### 2. Candidate Key

A **Candidate Key** is a minimal Super Key, meaning it is a set of attributes that uniquely identifies a row, and if you remove any attribute from it, it loses the unique identification property.

- **Definition:** A minimal Super Key. All values must be unique, and none can be NULL (non-nullability is enforced by the Primary Key constraint, which is chosen from a candidate key).
- **Example (in Employee table):**
  - {EmployeeID} (minimal set, unique)
  - {SSN} (minimal set, unique)
  - *If we remove EmployeeID from {EmployeeID, SSN}, it remains a Candidate Key, but a Candidate Key must be minimal. Since {EmployeeID} and {SSN} can individually identify a row, they are the minimal candidate keys.*

### 3. Primary Key

The **Primary Key (PK)** is a Candidate Key that the database designer chooses to uniquely identify each record in a table. It is the principal key used for referencing and ensuring entity integrity.

- **Properties:**
  - Must contain **unique** values.
  - Cannot contain **NULL** values (entity integrity constraint).
  - A table can have **only one** Primary Key.
- **Example (in Employee table):** The designer selects **EmployeeID** as the Primary Key.

### 4. Alternate Key

An **Alternate Key** is any Candidate Key that was *not* chosen as the Primary Key for the table.

- **Definition:** A Candidate Key that is not the Primary Key.
- **Example (in Employee table):** Since {EmployeeID} was chosen as the Primary Key, the remaining Candidate Key, {SSN}, is the Alternate Key.

### 5. Unique Key

A **Unique Key** (or Unique Constraint) is a key that ensures all values in a column or a set of columns are unique.

- **Properties:**
  - Must contain **unique** values.
  - Can contain **one NULL** value (unlike a Primary Key).
  - A table can have **multiple** Unique Keys.
- **Example (in Employee table):** SSN is also a Unique Key (in addition to being an Alternate Key).

### 6. Composite/Compound Key

A **Composite Key** (or Compound Key) is a key that consists of **two or more attributes** that together uniquely identify a row. It is used when a single column is not enough to guarantee uniqueness.

- **Example:** Consider an **Enrollment** table that records which students are in which courses:
  - {StudentID, CourseID}
  - Neither StudentID alone nor CourseID alone is unique in the Enrollment table (a student can take many courses, and a course can have many students). However, the *combination* of {**StudentID, CourseID**} is unique for a single enrollment record, making it the Composite Primary Key.

## 7. Foreign Key

A **Foreign Key (FK)** is a column or a set of columns in one table (the referencing table) that refers to the Primary Key (or Unique Key) in another table (the referenced table). It is used to establish and enforce a link between the data in two tables, ensuring **referential integrity**.

- **Definition:** A set of attributes in one table that references a key in another table.
- **Example (in Employee table): DeptID** is a Foreign Key. It links the Employee table to the Department table's Primary Key (let's assume it is DepartmentID). This ensures that every DeptID value in the Employee table must already exist in the Department table.

### Relational Model Notation

- Relation schema  $R$  of degree  $n$  is denoted by  $R(A_1, A_2, \dots, A_n)$ .
- Uppercase letters  $Q, R, S$  denote relation names.
- Lowercase letters  $q, r, s$  denote relation states.
- Letters  $t, u, v$  denote tuples.
- In general, the name of a relation schema such as STUDENT also indicates the current set of tuples in that relation.
- An attribute  $A$  can be qualified with the relation name  $R$  to which it belongs by using the dot notation  $R.A$  for example, STUDENT.Name or STUDENT.Age.
- An  $n$ -tuple  $t$  in a relation  $r(R)$  is represented as  $t = \langle v_1, v_2, \dots, v_n \rangle$  where  $v_i$  is the value corresponding to the attribute  $A_i$ . The value  $v_i$  for attribute  $A_i$  in tuple  $t$  can be accessed using  $t[A_i]$  or  $t.A_i$ .

### Characteristics of the Relational Model

- **Data Representation:** Data is organized in tables (relations), with rows (tuples) representing records and columns (attributes) representing data fields.
- **Atomic Values:** Each attribute in a table contains atomic values, meaning no multi-valued or nested data is allowed in a single cell.
- **Unique Keys:** Every table has a primary key to uniquely identify each record, ensuring no duplicate rows.
- **Attribute Domain:** Each attribute has a defined domain, specifying the valid data types and constraints for the values it can hold.
- **Data Independence:** The model ensures logical and physical data independence, allowing changes in the database schema without affecting the application layer.
- **Relational Operations:** Supports operations like selection, projection, join, union and intersection, enabling powerful data retrieval manipulation.
- **Data Consistency:** Ensures data consistency through constraints, reducing redundancy and anomalies.
- **Set-Based Representation:** Tables in the relational model are treated as sets and operations follow mathematical set theory principles.

### Constraints in Relational Model

While designing the Relational Model, we define some conditions which must hold for data present in the database are called Constraints. These constraints are checked before

performing any operation (insertion, deletion and updation) in the database. If there is a violation of any of the constraints, the operation will fail.

### 1. Domain Constraints

Domain Constraints ensure that the value of each attribute A in a tuple must be an atomic value derived from its specified domain, dom(A). Domains are defined by the data types associated with the attributes. Common data types include:

- **Numeric types:** Includes integers (short, regular and long) for whole numbers and real numbers (float, double-precision) for decimal values, allowing precise calculations.
- **Character types:** Consists of fixed-length (CHAR) and variable-length (VARCHAR, TEXT) strings for storing text data of various sizes.
- **Boolean values:** Stores true or false values, often used for flags or conditional checks in databases.
- **Specialized types:** Includes types for date (DATE), time (TIME), timestamp (TIMESTAMP) and money (MONEY), used for precise handling of time-related and financial data.

### 2. Key Integrity

Every relation in the database should have at least one set of attributes that defines a tuple uniquely. Those set of attributes is called keys. e.g.; ROLL\_NO in STUDENT is key. No two students can have the same roll number. So a key has two properties:

- It should be unique for all tuples.
- It can't have NULL values.

### 3. Referential Integrity Constraints

When one attribute of a relation can only take values from another attribute of the same relation or any other relation, it is called referential integrity. Let us suppose we have 2 relations

**Table: STUDENT**

| ROLL_NO | NAME   | ADDRESS | PHONE      | AGE | BRANCH_CODE |
|---------|--------|---------|------------|-----|-------------|
| 1       | RAM    | DELHI   | 9455123451 | 18  | CS          |
| 2       | RAMESH | GURGAON | 9652431543 | 18  | CS          |
| 3       | SUJIT  | ROHTAK  | 9156253131 | 20  | ECE         |
| 4       | SURESH | DELHI   |            | 18  | IT          |

**Table: BRANCH**

| BRANCH_CODE | BRANCH_NAME                               |
|-------------|-------------------------------------------|
| CS          | COMPUTER SCIENCE                          |
| IT          | INFORMATION TECHNOLOGY                    |
| ECE         | ELECTRONICS AND COMMUNICATION ENGINEERING |
| CV          | CIVIL ENGINEERING                         |

**Explanation:** BRANCH\_CODE of STUDENT can only take the values which are present in BRANCH\_CODE of BRANCH which is called referential integrity constraint. The relation which is referencing another relation is called REFERENCING RELATION (STUDENT in this case) and the relation to which other relations refer is called REFERENCED RELATION (BRANCH in this case).

### Codd Rules in Relational Model

E.F. Codd, the creator of the relational model, proposed 12 rules (known as Codd's 12 Rules) that define what constitutes a relational database system. These rules emphasize the importance of data independence, consistency and structure.

### Key Codd's Rules

- **Rule 1:** The information rule : All information in a relational database is represented logically in tables (relations).
- **Rule 2:** The guaranteed access rule : Every data element is accessible by using a combination of table name, primary key and attribute name.
- **Rule 5:** The powerful language rule : A relational DBMS should have a comprehensive language capable of expressing all relational queries.

### Advantages of the Relational Model

- **Simple model:** Relational Model is simple and easy to use in comparison to other languages.
- **Flexible:** Relational Model is more flexible than any other relational model present.
- **Secure:** Relational Model is more secure than any other relational model.
- **Data Accuracy:** Data is more accurate in the relational data model.
- **Data Integrity:** The integrity of the data is maintained in the relational model.
- **Operations can be Applied Easily:** It is better to perform operations in the relational model.

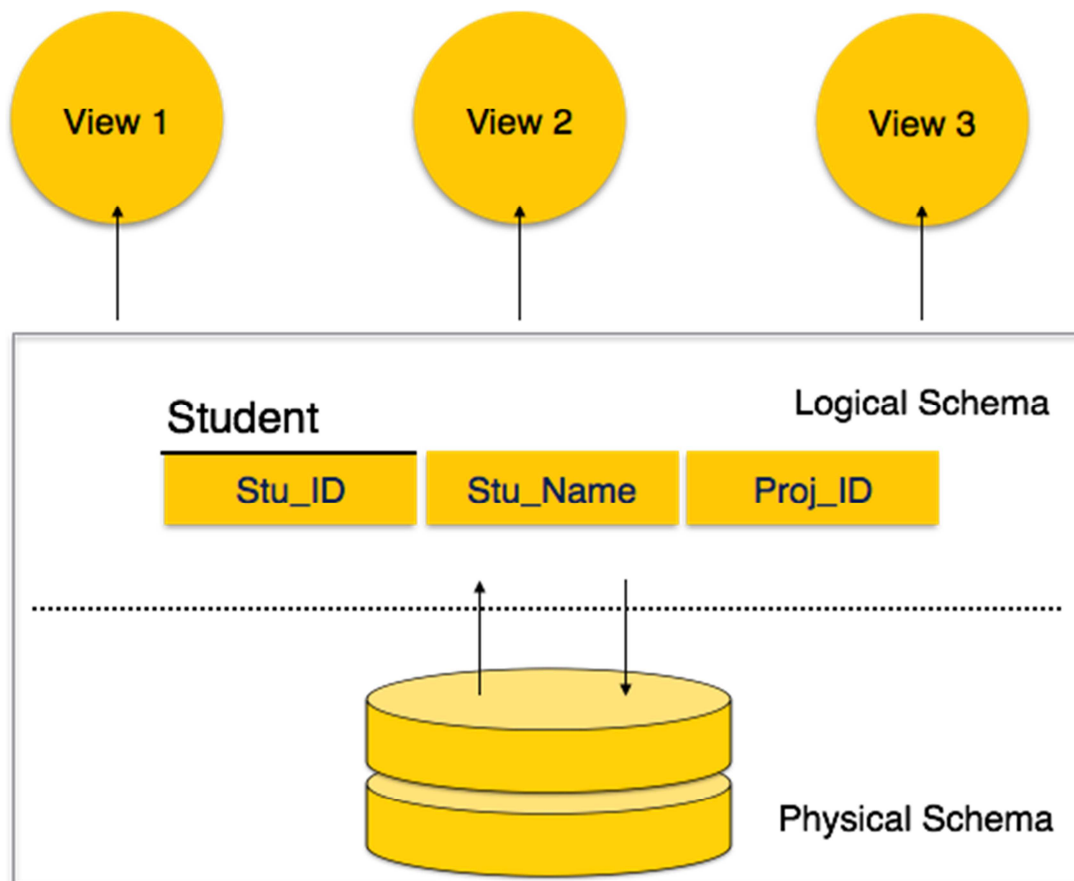
### Disadvantages of the Relational Model

- **Performance:** The relational model can experience performance issues with very large databases.
- **Complexity for Complex Data:** The model struggles with hierarchical or complex data relationships, which might be better handled with other models like the Graph or Document model.
- **Normalization Overhead:** Extensive use of normalization can result in complex queries and slower performance

### Database Schema and Schema diagram

A database schema is the skeleton structure that represents the logical view of the entire database. It defines how the data is organized and how the relations among them are associated. It formulates all the constraints that are to be applied on the data.

A database schema defines its entities and the relationship among them. It contains a descriptive detail of the database, which can be depicted by means of schema diagrams. Its the database designers who design the schema to help programmers understand the database and make it useful.



A database schema can be divided broadly into two categories –

- **Physical Database Schema** – This schema pertains to the actual storage of data and its form of storage like files, indices, etc. It defines how the data will be stored in a secondary storage.
- **Logical Database Schema** – This schema defines all the logical constraints that need to be applied on the data stored. It defines tables, views, and integrity constraints.

### Database Instance

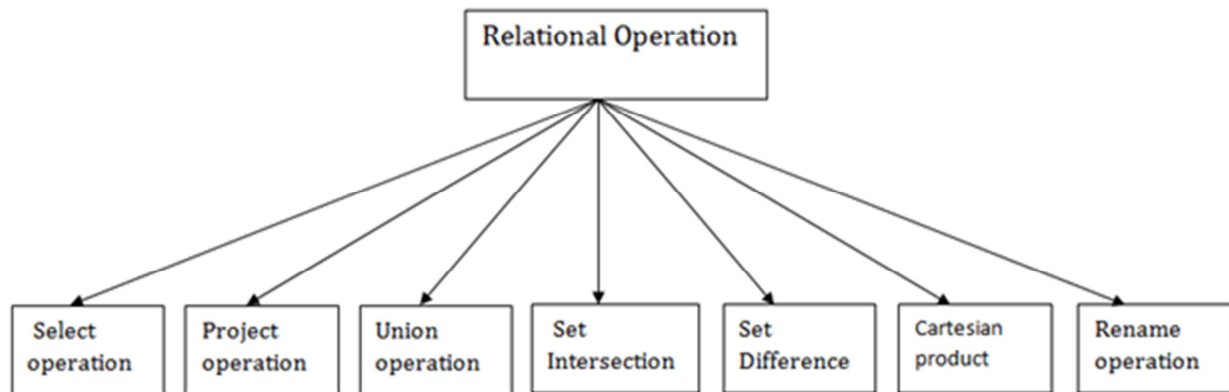
It is important that we distinguish these two terms individually. Database schema is the skeleton of database. It is designed when the database doesn't exist at all. Once the database is operational, it is very difficult to make any changes to it. A database schema does not contain any data or information.

A database instance is a state of operational database with data at any given time. It contains a snapshot of the database. Database instances tend to change with time. A DBMS ensures that its every instance (state) is in a valid state, by diligently following all the validations, constraints, and conditions that the database designers have imposed.

## Relational Algebra:

Relational algebra is a procedural query language. It gives a step by step process to obtain the result of the query. It uses operators to perform queries.

### Types of Relational operation



#### 1. Select Operation:

- The select operation selects tuples that satisfy a given predicate.
- It is denoted by sigma ( $\sigma$ ).
- Notation:  $\sigma p(r)$

#### Where:

$\sigma$  is used for selection prediction

$r$  is used for relation

$p$  is used as a propositional logic formula which may use connectors like: AND OR and NOT. These relational can use as relational operators like  $=, \neq, \geq, <, >, \leq$ .

#### For example: LOAN Relation

| BRANCH_NAME | LOAN_NO | AMOUNT |
|-------------|---------|--------|
| Downtown    | L-17    | 1000   |
| Redwood     | L-23    | 2000   |
| Perryride   | L-15    | 1500   |
| Downtown    | L-14    | 1500   |

|           |      |      |
|-----------|------|------|
| Mianus    | L-13 | 500  |
| Roundhill | L-11 | 900  |
| Perryride | L-16 | 1300 |

**Input:**

$\sigma$  BRANCH\_NAME="perryride" (LOAN)

**Output:**

| BRANCH_NAME | LOAN_NO | AMOUNT |
|-------------|---------|--------|
| Perryride   | L-15    | 1500   |
| Perryride   | L-16    | 1300   |

**2. Project Operation:**

- This operation shows the list of those attributes that we wish to appear in the result.  
Rest of the attributes are eliminated from the table.
- It is denoted by  $\Pi$ .

Notation:  $\Pi A_1, A_2, A_n (r)$

**Where**

**A1, A2, A3** is used as an attribute name of relation **r**.

**Example: CUSTOMER RELATION**

| NAME    | STREET  | CITY     |
|---------|---------|----------|
| Jones   | Main    | Harrison |
| Smith   | North   | Rye      |
| Hays    | Main    | Harrison |
| Curry   | North   | Rye      |
| Johnson | Alma    | Brooklyn |
| Brooks  | Senator | Brooklyn |

**Input:**

[[ NAME, CITY (CUSTOMER)

**Output:**

| NAME    | CITY     |
|---------|----------|
| Jones   | Harrison |
| Smith   | Rye      |
| Hays    | Harrison |
| Curry   | Rye      |
| Johnson | Brooklyn |
| Brooks  | Brooklyn |

**3. Union Operation:**

- Suppose there are two tuples R and S. The union operation contains all the tuples that are either in R or S or both in R & S.
- It eliminates the duplicate tuples. It is denoted by  $\cup$ .

Notation:  $R \cup S$

A union operation must hold the following condition:

- R and S must have the attribute of the same number.
- Duplicate tuples are eliminated automatically.

Example:

**DEPOSITOR RELATION**

| CUSTOMER_NAME | ACCOUNT_NO |
|---------------|------------|
| Johnson       | A-101      |
| Smith         | A-121      |
| Mayes         | A-321      |
| Turner        | A-176      |
| Johnson       | A-273      |
| Jones         | A-472      |
| Lindsay       | A-284      |

**BORROW RELATION**

| CUSTOMER_NAME | LOAN_NO |
|---------------|---------|
| Jones         | L-17    |
| Smith         | L-23    |
| Hayes         | L-15    |

|          |      |
|----------|------|
| Jackson  | L-14 |
| Curry    | L-93 |
| Smith    | L-11 |
| Williams | L-17 |

**Input:**

$\Pi$  CUSTOMER\_NAME (BORROW)  $\cup$   $\Pi$  CUSTOMER\_NAME (DEPOSITOR)

**Output:**

| CUSTOMER_NAME |
|---------------|
| Johnson       |
| Smith         |
| Hayes         |
| Turner        |
| Jones         |
| Lindsay       |
| Jackson       |
| Curry         |
| Williams      |
| Mayes         |

**4. Set Intersection:**

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in both R & S.
- It is denoted by intersection  $\cap$ .

Notation:  $R \cap S$

**Example:** Using the above DEPOSITOR table and BORROW table

**Input:**

$\Pi$  CUSTOMER\_NAME (BORROW)  $\cap$   $\Pi$  CUSTOMER\_NAME (DEPOSITOR)

**Output:**

| CUSTOMER_NAME |
|---------------|
| Smith         |
| Jones         |

## 5. Set Difference:

- Suppose there are two tuples R and S. The set intersection operation contains all tuples that are in R but not in S.
- It is denoted by intersection minus (-).

Notation:  $R - S$

**Example:** Using the above DEPOSITOR table and BORROW table

**Input:**

$[[ \text{CUSTOMER\_NAME (BORROW)} ] - [ \text{CUSTOMER\_NAME (DEPOSITOR)} ]$

**Output:**

| CUSTOMER_NAME |
|---------------|
| Jackson       |
| Hayes         |
| Willians      |
| Curry         |

## 6. Cartesian product

- The Cartesian product is used to combine each row in one table with each row in the other table. It is also known as a cross product.
- It is denoted by X.

Notation:  $E \times D$

Example:

**EMPLOYEE**

| EMP_ID | EMP_NAME | EMP_DEPT |
|--------|----------|----------|
| 1      | Smith    | A        |
| 2      | Harry    | C        |
| 3      | John     | B        |

**DEPARTMENT**

| DEPT_NO | DEPT_NAME |
|---------|-----------|
| A       | Marketing |
| B       | Sales     |
| C       | Legal     |

**Input:**

EMPLOYEE X DEPARTMENT

**Output:**

| EMP_ID | EMP_NAME | EMP_DEPT | DEPT_NO | DEPT_NAME |
|--------|----------|----------|---------|-----------|
| 1      | Smith    | A        | A       | Marketing |
| 1      | Smith    | A        | B       | Sales     |
| 1      | Smith    | A        | C       | Legal     |
| 2      | Harry    | C        | A       | Marketing |
| 2      | Harry    | C        | B       | Sales     |
| 2      | Harry    | C        | C       | Legal     |
| 3      | John     | B        | A       | Marketing |
| 3      | John     | B        | B       | Sales     |
| 3      | John     | B        | C       | Legal     |

**7. Rename Operation:**

The rename operation is used to rename the output relation. It is denoted by  **$\rho$**  ( $\rho$ ).

**Example:** We can use the rename operator to rename STUDENT relation to STUDENT1.

$\rho(\text{STUDENT1}, \text{STUDENT})$

*Note: Apart from these common operations Relational algebra can be used in Join operations.*

**Join Operations:**

A Join operation combines related tuples from different relations, if and only if a given join condition is satisfied. It is denoted by  $\bowtie$ .

Example:

**EMPLOYEE**

| EMP_CODE | EMP_NAME |
|----------|----------|
| 101      | Stephan  |
| 102      | Jack     |
| 103      | Harry    |

**SALARY**

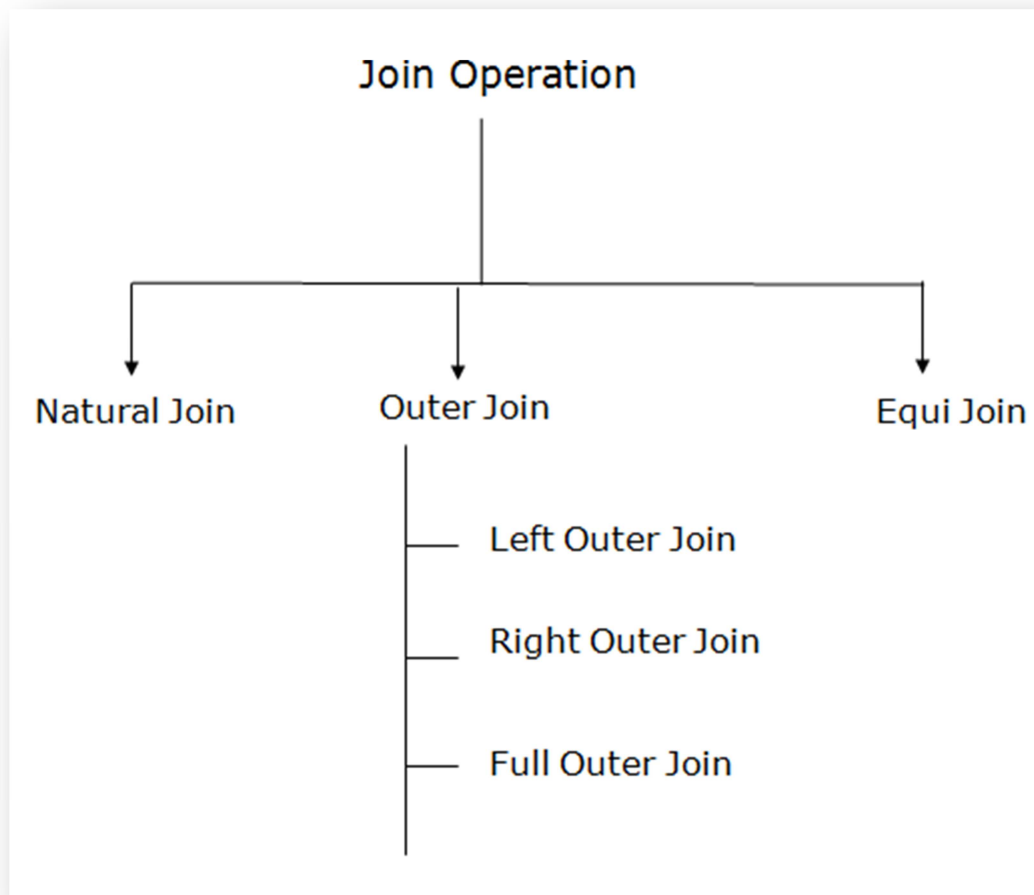
| EMP_CODE | SALARY |
|----------|--------|
| 101      | 50000  |
| 102      | 30000  |
| 103      | 25000  |

Operation: (EMPLOYEE  $\bowtie$  SALARY)

**Result:**

| EMP_CODE | EMP_NAME | SALARY |
|----------|----------|--------|
| 101      | Stephan  | 50000  |
| 102      | Jack     | 30000  |
| 103      | Harry    | 25000  |

### Types of Join operations:



### **1. Natural Join:**

- A natural join is the set of tuples of all combinations in R and S that are equal on their common attribute names.
- It is denoted by  $\bowtie$ .

**Example:** Let's use the above EMPLOYEE table and SALARY table:

**Input:**

$\Pi$ [EMP\_NAME, SALARY (EMPLOYEE  $\bowtie$  SALARY)]

**Output:**

| EMP_NAME | SALARY |
|----------|--------|
| Stephan  | 50000  |
| Jack     | 30000  |
| Harry    | 25000  |

## 2. Outer Join:

The outer join operation is an extension of the join operation. It is used to deal with missing information.

**Example:**

### EMPLOYEE

| EMP_NAME | STREET      | CITY      |
|----------|-------------|-----------|
| Ram      | Civil line  | Mumbai    |
| Shyam    | Park street | Kolkata   |
| Ravi     | M.G. Street | Delhi     |
| Hari     | Nehru nagar | Hyderabad |

### FACT\_WORKERS

| EMP_NAME | BRANCH  | SALARY |
|----------|---------|--------|
| Ram      | Infosys | 10000  |
| Shyam    | Wipro   | 20000  |
| Kuber    | HCL     | 30000  |
| Hari     | TCS     | 50000  |

**Input:**

(EMPLOYEE  $\bowtie$  FACT\_WORKERS)

**Output:**

| EMP_NAME | STREET      | CITY      | BRANCH  | SALARY |
|----------|-------------|-----------|---------|--------|
| Ram      | Civil line  | Mumbai    | Infosys | 10000  |
| Shyam    | Park street | Kolkata   | Wipro   | 20000  |
| Hari     | Nehru nagar | Hyderabad | TCS     | 50000  |

An outer join is basically of three types:

- Left outer join
- Right outer join
- Full outer join

**a. Left outer join:**

- Left outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In the left outer join, tuples in R have no matching tuples in S.
- It is denoted by  $\bowtie$ .

**Example:** Using the above EMPLOYEE table and FACT\_WORKERS table

**Input:** EMPLOYEE  $\bowtie$  FACT\_WORKERS

| EMP_NAME | STREET       | CITY      | BRANCH  | SALARY |
|----------|--------------|-----------|---------|--------|
| Ram      | Civil line   | Mumbai    | Infosys | 10000  |
| Shyam    | Park street  | Kolkata   | Wipro   | 20000  |
| Hari     | Nehru street | Hyderabad | TCS     | 50000  |
| Ravi     | M.G. Street  | Delhi     | NULL    | NULL   |

**b. Right outer join:**

- Right outer join contains the set of tuples of all combinations in R and S that are equal on their common attribute names.
- In right outer join, tuples in S have no matching tuples in R.
- It is denoted by  $\bowtie$ .

**Example:** Using the above EMPLOYEE table and FACT\_WORKERS Relation

**Input:** EMPLOYEE  $\bowtie$  FACT\_WORKERS

**Output:**

| EMP_NAME | BRANCH  | SALARY | STREET       | CITY      |
|----------|---------|--------|--------------|-----------|
| Ram      | Infosys | 10000  | Civil line   | Mumbai    |
| Shyam    | Wipro   | 20000  | Park street  | Kolkata   |
| Hari     | TCS     | 50000  | Nehru street | Hyderabad |
| Kuber    | HCL     | 30000  | NULL         | NULL      |

### c. Full outer join:

- Full outer join is like a left or right join except that it contains all rows from both tables.
- In full outer join, tuples in R that have no matching tuples in S and tuples in S that have no matching tuples in R in their common attribute name.
- It is denoted by  $\bowtie$ .

**Example:** Using the above EMPLOYEE table and FACT\_WORKERS table

**Input:** EMPLOYEE  $\bowtie$  FACT\_WORKERS

**Output:**

| EMP_NAME | STREET       | CITY      | BRANCH  | SALARY |
|----------|--------------|-----------|---------|--------|
| Ram      | Civil line   | Mumbai    | Infosys | 10000  |
| Shyam    | Park street  | Kolkata   | Wipro   | 20000  |
| Hari     | Nehru street | Hyderabad | TCS     | 50000  |
| Ravi     | M.G. Street  | Delhi     | NULL    | NULL   |
| Kuber    | NULL         | NULL      | HCL     | 30000  |

### 3. Equi join:

It is also known as an inner join. It is the most common join. It is based on matched data as per the equality condition. The equi join uses the comparison operator(=).

**Example:**

#### CUSTOMER RELATION

| CLASS_ID | NAME    |
|----------|---------|
| 1        | John    |
| 2        | Harry   |
| 3        | Jackson |

#### PRODUCT

| PRODUCT_ID | CITY  |
|------------|-------|
| 1          | Delhi |

|   |        |
|---|--------|
| 2 | Mumbai |
| 3 | Noida  |

**Input:** CUSTOMER  $\bowtie$  PRODUCT

**Output:**

| CLASS_ID | NAME  | PRODUCT_ID | CITY   |
|----------|-------|------------|--------|
| 1        | John  | 1          | Delhi  |
| 2        | Harry | 2          | Mumbai |
| 3        | Harry | 3          | Noida  |

### **Relational Calculus**

- There is an alternate way of formulating queries known as Relational Calculus. Relational calculus is a non-procedural query language.
- In the non-procedural query language, the user is concerned with the details of how to obtain the end results.
- The relational calculus tells what to do but never explains how to do.
- Most commercial relational languages are based on aspects of relational calculus.

### **Why it is called Relational Calculus?**

- It is based on Predicate calculus, a name derived from branch of symbolic language. A predicate is a truth-valued function with arguments.
- On substituting values for the arguments, the function result in an expression called a proposition. It can be either true or false.
- It is a tailored version of a subset of the Predicate Calculus to communicate with the relational database.

**Many of the calculus expressions involves the use of Quantifiers. There are two types of quantifiers:**

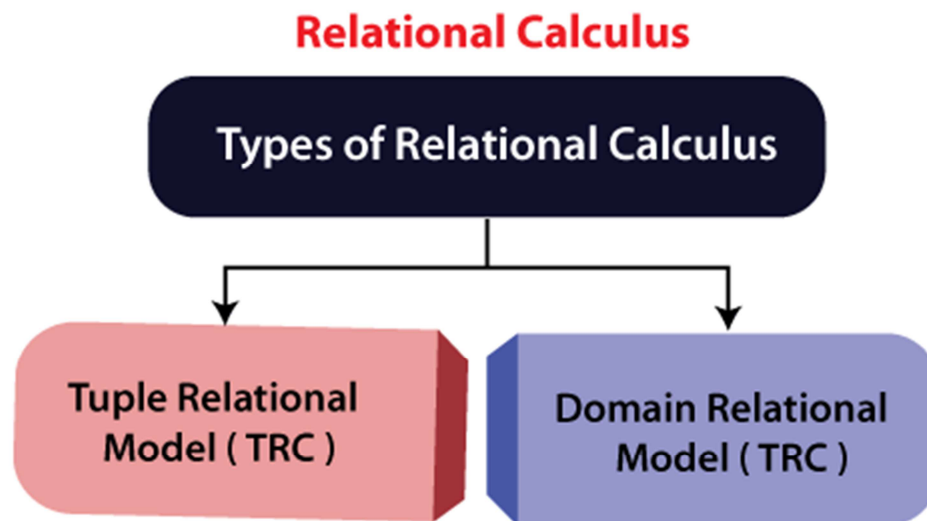
- **Universal Quantifiers:** The universal quantifier denoted by  $\forall$  is read as for all which means that in a given set of tuples exactly all tuples satisfy a given condition.
- **Existential Quantifiers:** The existential quantifier denoted by  $\exists$  is read as for all which means that in a given set of tuples there is at least one occurrences whose value satisfy a given condition.

Before using the concept of quantifiers in formulas, we need to know the concept of Free and Bound Variables.

A tuple variable  $t$  is bound if it is quantified which means that if it appears in any occurrences a variable that is not bound is said to be free.

Free and bound variables may be compared with global and local variable of programming languages.

Types of Relational calculus:



### 1. Tuple Relational Calculus (TRC)

It is a non-procedural query language which is based on finding a number of tuple variables also known as range variable for which predicate holds true. It describes the desired information without giving a specific procedure for obtaining that information. The tuple relational calculus is specified to select the tuples in a relation. In TRC, filtering variable uses the tuples of a relation. The result of the relation can have one or more tuples.

#### Notation:

A Query in the tuple relational calculus is expressed as following notation

1.  $\{T \mid P(T)\}$  or  $\{T \mid \text{Condition}(T)\}$

Where

**T** is the resulting tuples

**P(T)** is the condition used to fetch T.

**For example:**

$\{ T.\text{name} \mid \text{Author}(T) \text{ AND } T.\text{article} = \text{'database'} \}$

**Output:** This query selects the tuples from the AUTHOR relation. It returns a tuple with 'name' from Author who has written an article on 'database'.

TRC (tuple relation calculus) can be quantified. In TRC, we can use Existential ( $\exists$ ) and Universal Quantifiers ( $\forall$ ).

**For example:**

$\{ R \mid \exists T \in \text{Authors}(T.\text{article} = \text{'database'} \text{ AND } R.\text{name} = T.\text{name}) \}$

**Output:** This query will yield the same result as the previous one.

Tuple Relational Calculus is a non-procedural query language unlike relational algebra. Tuple Calculus provides only the description of the query but it does not provide the methods to solve it. Thus, it explains what to do but not how to do. In Tuple Calculus, a query is expressed as  $\{t \mid P(t)\}$

where  $t$  = resulting tuples,

$P(t)$  = known as Predicate and these are the conditions that are used to fetch  $t$

Thus, it generates set of all tuples  $t$ , such that Predicate  $P(t)$  is true for  $t$ .

$P(t)$  may have various conditions logically combined with OR ( $\vee$ ), AND ( $\wedge$ ), NOT ( $\neg$ ).

It also uses quantifiers:

$\exists t \in r (Q(t))$  = "there exists" a tuple in  $t$  in relation  $r$  such that predicate  $Q(t)$  is true.

$\forall t \in r (Q(t))$  =  $Q(t)$  is true "for all" tuples in relation  $r$ .

**Example:**

**Table-1: Customer**

| Customer name | Street | City      |
|---------------|--------|-----------|
| Saurabh       | A7     | Patiala   |
| Mehak         | B6     | Jalandhar |
| Sumiti        | D9     | Ludhiana  |
| Ria           | A5     | Patiala   |

**Table-2: Branch**

| Branch name | Branch city |
|-------------|-------------|
| ABC         | Patiala     |
| DEF         | Ludhiana    |
| GHI         | Jalandhar   |

**Table-3: Account**

| Account number | Branch name | Balance |  |
|----------------|-------------|---------|--|
| 1111           | ABC         | 50000   |  |
| 1112           | DEF         | 10000   |  |
| 1113           | GHI         | 9000    |  |
| 1114           | ABC         | 7000    |  |

**Table-4: Loan**

| Loan number | Branch name | Amount |
|-------------|-------------|--------|
| L33         | ABC         | 10000  |
| L35         | DEF         | 15000  |
| L49         | GHI         | 9000   |
| L98         | DEF         | 65000  |

**Table-5: Borrower**

| Customer name | Loan number |
|---------------|-------------|
| Saurabh       | L33         |
| Mehak         | L49         |
| Ria           | L98         |

**Table-6: Depositor**

| Customer name | Account number |
|---------------|----------------|
| Saurabh       | 1111           |
| Mehak         | 1113           |
| Sumiti        | 1114           |

**Queries-1:** Find the loan number, branch, amount of loans of greater than or equal to 10000 amount.

$\{t \mid t \in \text{loan} \wedge t[\text{amount}] \geq 10000\}$

Resulting relation:

| Loan number | Branch name | Amount |
|-------------|-------------|--------|
| L33         | ABC         | 10000  |
| L35         | DEF         | 15000  |
| L98         | DEF         | 65000  |

In the above query,  $t[\text{amount}]$  is known as tuple variable.

**Queries-2:** Find the loan number for each loan of an amount greater or equal to 10000.

$\{t \mid \exists s \in \text{loan}(t[\text{loan number}] = s[\text{loan number}]$   
 $\wedge s[\text{amount}] \geq 10000)\}$

Resulting relation:

| Loan number |
|-------------|
| L33         |
| L35         |
| L98         |

**Queries-3:** Find the names of all customers who have a loan and an account at the bank.

$\{t \mid \exists s \in \text{borrower}(t[\text{customer-name}] = s[\text{customer-name}])\}$

$\wedge \exists u \in \text{depositor}(t[\text{customer-name}] = u[\text{customer-name}]))\}$

Resulting relation:

| Customer name |
|---------------|
| Saurabh       |
| Mehak         |

**Queries-4:** Find the names of all customers having a loan at the “ABC” branch.

$\{t \mid \exists s \in \text{borrower}(t[\text{customer-name}] = s[\text{customer-name}])\}$

$\wedge \exists u \in \text{loan}(u[\text{branch-name}] = \text{“ABC”} \wedge u[\text{loan-number}] = s[\text{loan-number}]))\}$

Resulting relation:

| Customer name |
|---------------|
| Saurabh       |

## 2. Domain Relational Calculus (DRC)

- The second form of relation is known as Domain relational calculus.
- In domain relational calculus, filtering variable uses the domain of attributes. Domain relational calculus uses the same operators as tuple calculus. It uses logical connectives  $\wedge$  (and),  $\vee$  (or) and  $\neg$  (not). It uses Existential ( $\exists$ ) and Universal Quantifiers ( $\forall$ ) to bind the variable. The QBE or Query by example is a query language related to domain relational calculus.

**Notation:**

$\{a_1, a_2, a_3, \dots, a_n \mid P(a_1, a_2, a_3, \dots, a_n)\}$

Where

**a1, a2** are attributes

**P** stands for formula built by inner attributes

**For example:**

$\{ \langle \text{article, page, subject} \rangle \mid \in \text{Books} \wedge \text{subject} = \text{'database'} \}$

**Output:** This query will yield the article, page, and subject from the relational Books, where the subject is a database.

**Domain Relational Calculus** is a non-procedural query language equivalent in power to Tuple Relational Calculus. Domain Relational Calculus provides only the description of the

query but it does not provide the methods to solve it. In Domain Relational Calculus, a query is expressed as,

$$\{ \langle x_1, x_2, x_3, \dots, x_n \rangle \mid P(x_1, x_2, x_3, \dots, x_n) \}$$

where,  $\langle x_1, x_2, x_3, \dots, x_n \rangle$  represents resulting domains variables and  $P(x_1, x_2, x_3, \dots, x_n)$  represents the condition or formula equivalent to the Predicate calculus.

### Predicate Calculus Formula:

1. Set of all comparison operators
2. Set of connectives like and, or, not
3. Set of quantifiers

### Example:

**Table-1: Customer**

| Customer name | Street     | City       |
|---------------|------------|------------|
| Debomit       | Kadamtala  | Alipurduar |
| Sayantana     | Udaypur    | Balurghat  |
| Soumya        | Nutanchati | Bankura    |
| Ritu          | Juhu       | Mumbai     |

**Table-2: Loan**

| Loan number | Branch name | Amount |
|-------------|-------------|--------|
| L01         | Main        | 200    |
| L03         | Main        | 150    |
| L10         | Sub         | 90     |
| L08         | Main        | 60     |

**Table-3: Borrower**

| Customer name | Loan number |
|---------------|-------------|
| Ritu          | L01         |
| Debomit       | L08         |
| Soumya        | L03         |

**Query-1:** Find the loan number, branch, amount of loans of greater than or equal to 100 amount.

$$\{ \langle l, b, a \rangle \mid \langle l, b, a \rangle \in \text{loan} \wedge (a \geq 100) \}$$

Resulting relation:

| Loan number | Branch name | Amount |
|-------------|-------------|--------|
| L01         | Main        | 200    |
| L03         | Main        | 150    |

**Query-2:** Find the loan number for each loan of an amount greater or equal to 150.

$$\{ \langle l \rangle \mid \exists b, a (\langle l, b, a \rangle \in \text{loan} \wedge (a \geq 150)) \}$$

Resulting relation:

| Loan number |
|-------------|
| L01         |
| L03         |

**Query-3:** Find the names of all customers having a loan at the “Main” branch and find the loan amount .

$\{ \langle c, a \rangle \mid \exists l (\langle c, l \rangle \in \text{borrower} \wedge \exists b (\langle l, b, a \rangle \in \text{loan} \wedge (b = \text{“Main”}))) \}$

Resulting relation:

| Customer Name | Amount |
|---------------|--------|
| Ritu          | 200    |
| Debomit       | 60     |
| Soumya        | 150    |

**Note:**

The domain variables those will be in resulting relation must appear before | within  $\langle$  and  $\rangle$  and all the domain variables must appear in which order they are in original relation or table.

### GTU Asked Questions: Practice Relational algebra

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|----|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. | List the relational algebra operators. Discuss any two such algebra operator with suitable example.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| 2. | <p>The relational database schema is given below.<br/> employee (person-name, street, city)<br/> works (person-name, company-name, salary)<br/> company (company-name, city)<br/> manages (person-name, manager-name)<br/> Write the relational algebra expressions for the given queries.<br/> 1.Find the names of all employees who work for First Bank Corporation.</p> $\pi_{\text{person-name}}(\sigma_{\text{company-name}=\text{'First Bank Corporation'}}(W))$ <p>Ans:<br/> 2.Find the names and cities of residence of all employees who work for First Bank Corporation.</p> $\pi_{\text{person-name, city}}(\sigma_{\text{company-name}=\text{'First Bank Corporation'}}(W) \bowtie E)$ <p>Ans:<br/> 3.. Find the names, street address, and cities of residence of all employees who work for First Bank Corporation and earn more than \$10,000 per annum.</p> <p>Ans:</p> $\pi_{\text{person-name, street, city}}(\sigma_{\text{company-name}=\text{'First Bank Corporation'} \wedge \text{salary} > 10000}(W) \bowtie E)$ |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | <p>4. Find the names of all employees in this database who do not work for First Bank Corporation.</p> $\pi_{\text{person-name}}(E) - \pi_{\text{person-name}}(\sigma_{\text{company-name}='First Bank Corporation'}(W))$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 3. | Describe the differences in meaning between the terms relation and relation schema.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 4. | <p>Write the following queries in relational algebra:</p> <p>(1) Find the names of suppliers who supply some red part.</p> <p>(2) Find the IDs of suppliers who supply some red or green part.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| 5. | Describe the Cartesian Product operation in relational algebra.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
| 6. | <p>Consider the relational database given below. Give an expression in the relational algebra to express each of the following queries:</p> <p>Employee (person-name, street, city),<br/> Works (person-name, company-name, salary),<br/> Company (company-name, city),<br/> Manages (person-name, manager-name)</p> <p>(1) Find name of all employees.</p> $\pi_{\text{person-name}}(\text{Employee})$ <p>(2) Find city of employee whose name is 'jashu'.</p> $\pi_{\text{city}}(\sigma_{\text{person-name}='jashu'}(\text{Employee}))$ <p>(3) Find name and city of all employees who are having salary &gt; 50000.</p> $\pi_{\text{person-name,city}}((\sigma_{\text{salary}>50000}(\text{Works})) \bowtie \text{Employee})$ <p>(4) Find total salary of all employees who are working for company 'HCL'.</p> $\mathcal{G}_{\text{SUM}(\text{salary}) \rightarrow \text{total-salary}}(\sigma_{\text{company-name}='HCL'}(\text{Works}))$ |
| 7. | <p>List the type of joins in relational algebra.</p> <p>Explain with example.</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |
| 8. | List unary relational operators and explain with example.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| 9. | <p>Consider the following relational database schema consisting of the four relation schemas:</p> <p><b>passenger</b> ( pid, pname, pgender, pcity)<br/> <b>agency</b> ( aid, aname, acity)<br/> <b>flight</b> (fid, fdate, time, src, dest)<br/> <b>booking</b> (pid, aid, fid, fdate)</p> <p>Answer the following questions using relational algebra queries.</p> <p>a. Get the details about all flights from Chennai to New Delhi.</p> $\sigma_{\text{src}='Chennai' \wedge \text{dest}='New Delhi'}(\text{flight})$ <p>b. Get the complete details of all flights to New Delhi.</p> $\sigma_{\text{dest}='New Delhi'}(\text{flight})$ <p>c. Find the passenger names for passengers who have bookings on at least one flight.</p>                                                                                                                                                                                                        |

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|    | $\pi_{pname}(\text{passenger} \bowtie (\pi_{pid}(\text{booking})))$                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| 10 | <p>Write Relational Algebra syntax for the following queries.</p> <p>Employee(eno,ename,salary,designation)<br/> Customer(cno,cname,address,city)</p> <p>1) Find out name of employees who are 'Manager'.</p> $\pi_{ename}(\sigma_{\text{designation}='Manager'}(\text{Employee}))$ <p>2) Display name of customers.</p> $\pi_{cname}(\text{Customer})$ <p>3) Retrieve Employee records whose salary is less than 20,000.</p> $\sigma_{\text{salary}<20000}(\text{Employee})$                                                                                                                 |
| 11 | <p>Write Relational Algebra syntax for the given queries using the following database.</p> <p>Employee(eno,ename,salary,designation)<br/> Customer(cno,cname,address,city)</p> <p>1) Find out name of employees who are also customers.</p> $\pi_{ename}(\text{Employee}) \cap \pi_{cname}(\text{Customer})$ <p>2) Find out name of person who are employees but not customers.</p> $\pi_{ename}(\text{Employee}) \setminus \pi_{cname}(\text{Customer})$ <p>3) Display all names who are either employees or customers.</p> $\pi_{ename}(\text{Employee}) \cup \pi_{cname}(\text{Customer})$ |